SOLID Principles

Episode 1 : The Setup

Let’s imagine a world…

You are hired as a Backend software Engineer at AlgoZenith.

You are happy that you finally landed a Software Engineer.

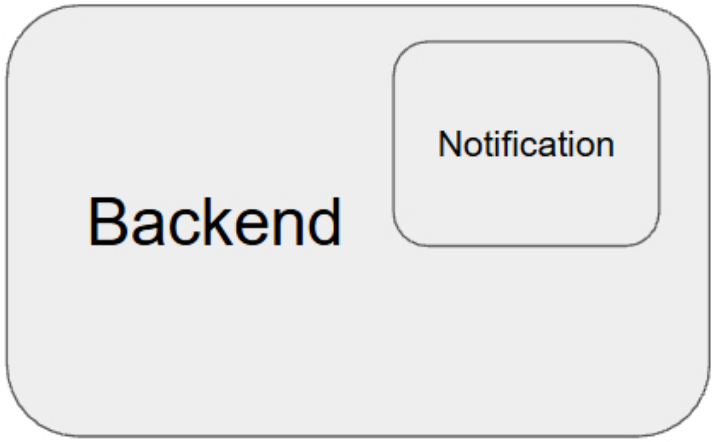
You are onboarded on backend codebase of AlgoZenith.

Right now Thousands of students are using AlgoZenith to Study DSA,LLD .. etc, but they are **not getting any reminder to complete the course content** on a periodic basis. To keep them regular, we want to add this.

Rohan, the CTO, has given you a starter project to implement this Notification Service, that is like a independent code that can be scaled.

What are we building

Implement a single class **NotificationService** with a method to send either an email or SMS based on a string parameter.




```cpp
struct User {
    std::string name;
    std::string email;
    std::string phone;
};

class NotificationService {
public:
    // Send a message to the user via the specified type ("email" or "sms")
    void sendNotification(const User& user, const std::string& message, const std::string& type) {

    }
};
```

What you do (as a fresh joinee)

```
class NotificationService {
public:
    // Send a message to the user via the specified type ("email" or "sms")
    void sendNotification(const User& user, const std::string& message, const
std::string& type) {
        if (type == "email") {
            // Simulate sending an email
            std::cout << "Email to " << user.email << ": " << message << "\n";
        } else if (type == "sms") {
            // Simulate sending an SMS
            std::cout << "SMS to " << user.phone << ": " << message << "\n";
        }
    }
};

int main() {
    User alice{"Alice", "alice@example.com", "1234567890"};
    NotificationService service;
    service.sendNotification(alice, "Welcome, Alice!", "email");
    service.sendNotification(alice, "Your order has shipped.", "sms");
}
```

The Pain of Growth starts…

Suppose the application needs to **support additional notification channels** like push notifications or in-app messages.

Or perhaps we need to

- **log notifications**,
- **format messages differently,**
- or change **how we decide which channel to use**.

Violation of Single Responsibility

```cpp
class NotificationService {
public:
    // Send a message to the user via the specified type ("email" or "sms")
    void sendNotification(const User& user, const std::string& message, const std::string& type) {
        if (type == "email") {
            // Simulate sending an email
            std::cout << "Email to " << user.email << ": " << message << "\n";
        } else if (type == "sms") {
            // Simulate sending an SMS
            std::cout << "SMS to " << user.phone << ": " << message << "\n";
        }
    }
};

int main() {
    User alice{"Alice", "alice@example.com", "1234567890"};
    NotificationService service;
    service.sendNotification(alice, "Welcome, Alice!", "email");
    service.sendNotification(alice, "Your order has shipped.", "sms");
}
```

Tight Coupling and Rigidity

```
10 class NotificationService {
11 public:
12     // Send a message to the user via the specified type ("email" or "sms")
13     void sendNotification(const User& user, const std::string& message, const
   std::string& type) {
14         if (type == "email") {
15             // Simulate sending an email
16             std::cout << "Email to " << user.email << ": " << message << "\n";
17         } else if (type == "sms") {
18             // Simulate sending an SMS
19             std::cout << "SMS to " << user.phone << ": " << message << "\n";
20         }
21     }
22 };
23
24 int main() {
25     User alice{"Alice", "alice@example.com", "1234567890"};
26     NotificationService service;
27     service.sendNotification(alice, "Welcome, Alice!", "email");
28     service.sendNotification(alice, "Your order has shipped.", "sms");
29 }
```

Scalability Issues (Class Explosion)

```cpp
class NotificationService {
public:
    // Send a message to the user via the specified type ("email" or "sms")
    void sendNotification(const User& user, const std::string& message, const std::string& type) {
        if (type == "email") {
            // Simulate sending an email
            std::cout << "Email to " << user.email << ": " << message << "\n";
        } else if (type == "sms") {
            // Simulate sending an SMS
            std::cout << "SMS to " << user.phone << ": " << message << "\n";
        }
    }
};

int main() {
    User alice{"Alice", "alice@example.com", "1234567890"};
    NotificationService service;
    service.sendNotification(alice, "Welcome, Alice!", "email");
    service.sendNotification(alice, "Your order has shipped.", "sms");
}
```

Difficulty in Testing and Maintenance

```cpp
class NotificationService {
public:
    // Send a message to the user via the specified type ("email" or "sms")
    void sendNotification(const User& user, const std::string& message, const std::string& type) {
        if (type == "email") {
            // Simulate sending an email
            std::cout << "Email to " << user.email << ": " << message << "\n";
        } else if (type == "sms") {
            // Simulate sending an SMS
            std::cout << "SMS to " << user.phone << ": " << message << "\n";
        }
    }
};

int main() {
    User alice{"Alice", "alice@example.com", "1234567890"};
    NotificationService service;
    service.sendNotification(alice, "Welcome, Alice!", "email");
    service.sendNotification(alice, "Your order has shipped.", "sms");
}
```

In short,

the initial design works, but it's fragile and not future-proof. We need to refactor the code to follow better design practices. A set of guidelines known as the SOLID principles can help us address these issues. SOLID stands for:

S – Single-Responsibility Principle

O – Open-Closed Principle

L – Liskov Substitution Principle

I – Interface Segregation Principle

D – Dependency Inversion Principle

In Next Episode -

Let’s start with the first principle, SRP, and see how it can make our design better.

SOLID Principles

Episode 2 - SRP Principle

Recap - Needs some SOLID changes

```cpp
class NotificationService {
public:
    // Send a message to the user via the specified type ("email" or "sms")
    void sendNotification(const User& user, const std::string& message, const std::string& type) {
        if (type == "email") {
            // Simulate sending an email
            std::cout << "Email to " << user.email << ": " << message << "\n";
        } else if (type == "sms") {
            // Simulate sending an SMS
            std::cout << "SMS to " << user.phone << ": " << message << "\n";
        }
    }
};

int main() {
    User alice{"Alice", "alice@example.com", "1234567890"};
    NotificationService service;
    service.sendNotification(alice, "Welcome, Alice!", "email");
    service.sendNotification(alice, "Your order has shipped.", "sms");
}
```

What is SRP ??

The Single-Responsibility Principle states that a class should have one, and only one, reason to change. In other words, a class should only do one job. If a class is handling multiple tasks, those tasks should be split into separate classes, each with a clear focus.

Our code ??

it contained logic for sending emails and sending SMS. This means changes in either email or SMS functionality would require changes to NotificationService.

Let’s refactor our design according to SRP:

EmailSender – a class that knows how to send an email to a user.

SMSSender – a class that knows how to send an SMS to a user.

NotificationService – now this class will delegate the work to the appropriate sender. It will decide which sender to use (email or SMS) but will not handle the low-level sending details itself.

This separation means **EmailSender** and **SMSSender** can evolve independently. For example, if tomorrow we change the way emails are sent (say, use a different SMTP library), we only modify EmailSender.

What is SRP ??

The Single-Responsibility Principle states that a class should have one, and only one, reason to change. In other words, a class should only do one job. If a class is handling multiple tasks, those tasks should be split into separate classes, each with a clear focus.

Our code ??

it contained logic for sending emails and sending SMS. This means changes in either email or SMS functionality would require changes to NotificationService.

We will create 2 separate classes !

```cpp
class EmailSender {
public:
    void sendEmail(const User& user, const std::string& message) {
        // Only handles email-sending details
        std::cout << "[Email] To: " << user.email
                  << " | Message: " << message << "\n";
    }
};

class SMSSender {
public:
    void sendSMS(const User& user, const std::string& message) {
        // Only handles SMS-sending details
        std::cout << "[SMS] To: " << user.phone
                  << " | Message: " << message << "\n";
    }
};
```

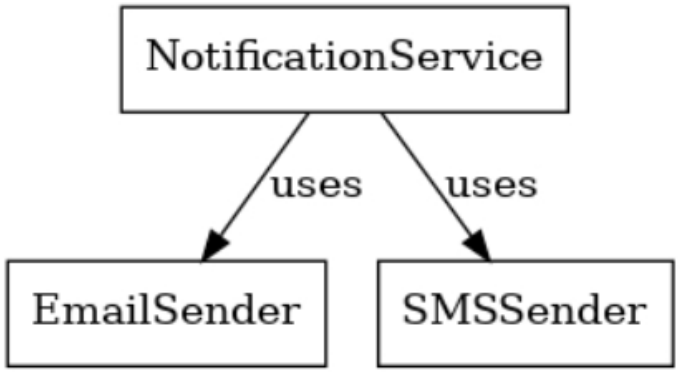
And delegate as needed !!

```
18
19 class NotificationService {
20 private:
21     EmailSender emailSender;
22     SMSSender smsSender;
23 public:
24     void sendNotification(const User& user, const std::string& message,
   const std::string& type) {
25         // Delegates the work to the appropriate sender
26         if (type == "email") {
27             emailSender.sendEmail(user, message);
28         } else if (type == "sms") {
29             smsSender.sendSMS(user, message);
30         }
31     }
32 };
```

Isn’t it just better !!

Now each class has a clear responsibility. EmailSender handles only emails, and SMSSender handles only SMS. The NotificationService is simpler – it just decides which sender to use and delegates the call.

After applying SRP, the notification system is split into specialized classes. The NotificationService uses EmailSender and SMSSender to handle the details of each channel. Each class has a single responsibility (one job), which makes the code easier to maintain and extend.



But are we really done ??

```
18
19 class NotificationService {
20 private:
21     EmailSender emailSender;
22     SMSSender smsSender;
23 public:
24     void sendNotification(const User& user, const std::string& message,
   const std::string& type) {
25         // Delegates the work to the appropriate sender
26         if (type == "email") {
27             emailSender.sendEmail(user, message);
28         } else if (type == "sms") {
29             smsSender.sendSMS(user, message);
30         }
31     }
32 };
```

In Next Episode -

How to remove the “IF ELSE” System

SOLID Principles

Episode 3 - Open-Closed Principle

But we were not done !!

```
18
19 class NotificationService {
20 private:
21     EmailSender emailSender;
22     SMSSender smsSender;
23 public:
24     void sendNotification(const User& user, const std::string& message,
   const std::string& type) {
25         // Delegates the work to the appropriate sender
26         if (type == "email") {
27             emailSender.sendEmail(user, message);
28         } else if (type == "sms") {
29             smsSender.sendSMS(user, message);
30         }
31     }
32 };
```

Open-Closed Principle (OCP)

The Open-Closed Principle says that classes and other entities should be open for extension, but closed for modification.

In practice, this means we should design our classes so that we can add new functionality (extend them) without changing the existing code.

We often achieve this by using abstraction and polymorphism

Looking at our NotificationService

adding a new notification type currently requires changing its code (not closed for modification). We can apply OCP by introducing an abstraction for "notification sending".

```
class NotificationService {
private:
    EmailSender emailSender;
    SMSSender smsSender;
public:
    void sendNotification(const User& user, const std::string& message,
const std::string& type) {
        // Delegates the work to the appropriate sender
        if (type == "email") {
            emailSender.sendEmail(user, message);
        } else if (type == "sms") {
            smsSender.sendSMS(user, message);
        }
    }
};
```

Plan:

1. Define an abstract base class or interface (in C++ terms, an abstract class with a pure virtual function) for senders, say `INotifier`, with a method like `send(const User&, const std::string&)`.

2. Have `EmailSender` and `SMSSender` (and any future senders) inherit from `INotifier` and implement the `send` method.

3. Change `NotificationService` to use `INotifier` references or pointers instead of concrete `EmailSender`/`SMSSender`. For example, `NotificationService` might have a method to accept any `INotifier` and invoke its `send` method, or maintain a list of `INotifier` objects to send through all available channels.

```cpp
// Abstract base class (interface) for notification senders
struct INotifier {
    virtual ~INotifier() = default;
    virtual void send(const User& user, const std::string& message) = 0;
};

class EmailSender : public INotifier {
public:
    void send(const User& user, const std::string& message) override {
        std::cout << "[Email] To: " << user.email << " | Message: " << message << "\n";
    }
};

class SMSSender : public INotifier {
public:
    void send(const User& user, const std::string& message) override {
        std::cout << "[SMS] To: " << user.phone << " | Message: " << message << "\n";
    }
};
```

And now we can do something like this …

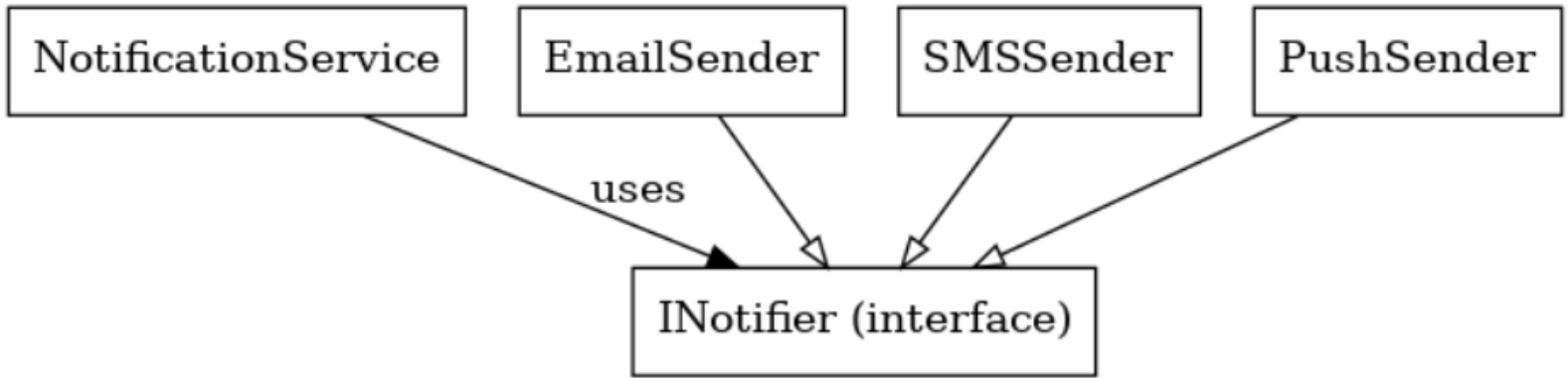
```
1  class NotificationService {
2  public:
3      // Send a notification using any notifier (polymorphism)
4      void sendViaChannel(INotifier& notifier, const User& user, const
   std::string& message) {
5          notifier.send(user, message);
6      }
7  };
8
9
10 User alice{"Alice", "alice@example.com", "1234567890"};
11 NotificationService service;
12 EmailSender emailSender;
13 SMSSender smsSender;
14
15 service.sendViaChannel(emailSender, alice, "Hello via Email!");
16 service.sendViaChannel(smsSender, alice, "Hello via SMS!");
17
```

Now thats some real progress !!

In a real application, you might choose the notifier based on configuration or user preference, but the key is that **NotificationService** doesn't need to change when a new notifier is introduced.

We can also imagine an enhanced **NotificationService** that holds a **list of INotifier*** and sends a message through all of them (broadcasting to multiple channels), but whether it sends through one or many channels, it treats them uniformly via the interface.

How to add new …



At this point, our design leverages two fundamental OOP concepts: inheritance and polymorphism.

adding a Push Notification sender would involve creating a PushSender class implementing INotifier::send. No changes to NotificationService are required;

We already did something that helps in future…

It’s worth noting that we also started to implement the **Dependency Inversion Principle** here (even if we haven’t discussed it yet): NotificationService is now depending on an abstraction (INotifier) rather than concrete classes.

We will formally discuss DIP soon, but first, let's ensure our design honors the next principle, Liskov Substitution, which is about making sure our inheritance actually makes sense.

SOLID Principles

Episode 4 - LSP

```cpp
// Abstract base class (interface) for notification senders
struct INotifier {
    virtual ~INotifier() = default;
    virtual void send(const User& user, const std::string& message) = 0;
};

class EmailSender : public INotifier {
public:
    void send(const User& user, const std::string& message) override {
        std::cout << "[Email] To: " << user.email << " | Message: " << message << "\n";
    }
};

class SMSSender : public INotifier {
public:
    void send(const User& user, const std::string& message) override {
        std::cout << "[SMS] To: " << user.phone << " | Message: " << message << "\n";
    }
};
```

We have a pretty good code aint it !!

```
class NotificationService {
public:
    // Send a notification using any notifier (polymorphism)
    void sendViaChannel(INotifier& notifier, const User& user, const
std::string& message) {
        notifier.send(user, message);
    }
};


User alice{"Alice", "alice@example.com", "1234567890"};
NotificationService service;
EmailSender emailSender;
SMSSender smsSender;

service.sendViaChannel(emailSender, alice, "Hello via Email!");
service.sendViaChannel(smsSender, alice, "Hello via SMS!");

```

Liskov Substitution Principle (LSP)

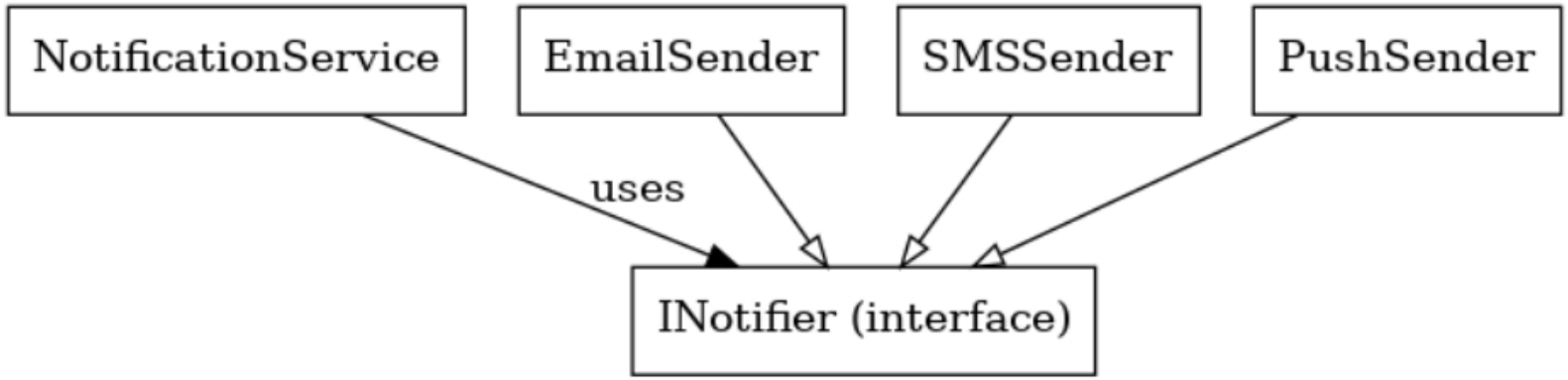
The Liskov Substitution Principle is perhaps the most theoretical of the SOLID principles. It states that subclasses should be substitutable for their base classes without altering the correctness of the program.

In simpler terms, wherever you use an interface or base class, any implementation or subclass of that interface should behave as expected and not break the expectations of the code that uses the base class.

In our context,

LSP means that any class that implements INotifier should truly fulfill the contract of INotifier::send in a way that doesn't surprise NotificationService (or whatever code is calling it).

For example, NotificationService expects that calling send(user, msg) will actually attempt to deliver msg to user in some fashion.



So what does that mean ??

If someone wrote a new notifier class that, say, ignored the message or required an extra setup step that NotificationService didn't know about, that would violate LSP because substituting that notifier would break the notification functionality.

Our current design is pretty straightforward regarding LSP: each INotifier subclass just sends the message. As long as they stick to that purpose, any INotifier can substitute for any other in sendViaChannel and things will work.

Some other example

```java
class Bird {
    public void fly() {
        System.out.println("Bird flying...");
    }
    public void eat() {
        System.out.println("Bird eating...");
    }
}

class Penguin extends Bird {
    public void fly() {
       throw new UnsupportedOperationException("Can't fly.");
    }
}

class LSPTest {
    public static void main(String[] args) {
        Bird bird = new Bird();
        bird.fly();
    }
}
```

```java
class Bird {
    public void eat() {
        System.out.println("Bird eating...");
    }
}

class FlightBird extends Bird {
    public void fly() {
        System.out.println("Bird flying...");
    }
}

class FlightlessBird extends Bird {

}
```

To ensure LSP in practice:

1. Keep interfaces focused and minimal (which ties into the next principle, ISP) so that implementing classes can fulfill them easily.
2. Document any expectations. For example, if INotifier::send expects that the User has certain contact info available (email for EmailSender, phone for SMSSender), those expectations should be clear. In our design, we assume User.email is valid when using EmailSender, etc., which is reasonable.
3. Write unit tests using the base class interface and run them against all subclasses to ensure none of the subclasses violate the expected behavior.

Ok… simple right ??

For now, we'll assume our notifiers follow LSP. Each notifier can be used wherever an INotifier is needed, and the NotificationService will successfully send notifications through it. The next principle, Interface Segregation, will further help ensure our interfaces remain easy to implement correctly.

SOLID Principles

Episode 5 - ISP

Interface Segregation Principle (ISP)

The Interface Segregation Principle advises that “no client should be forced to depend on methods it does not use”.

Put differently, it’s better to have many small, specific interfaces than a few monolithic ones that try to do everything.

For us,

In our notification system, INotifier has just one method send(...). This interface is already as segregated as it can get – each notifier class implements exactly what it needs. We don't have a case where, say, EmailSender is forced to implement an unrelated method. So our design for notifiers already follows ISP.

```
class NotificationService {
public:
    // Send a notification using any notifier (polymorphism)
    void sendViaChannel(INotifier& notifier, const User& user, const
std::string& message) {
        notifier.send(user, message);
    }
};

```

However to illustrate,

However, to illustrate why ISP matters, consider a scenario where we weren't so careful. Imagine we had designed a single interface INotificationService that included all possible notification-related actions, for example:

```
struct INotificationService {
    virtual void sendEmail(const User&, const std::string&) = 0;
    virtual void sendSMS(const User&, const std::string&) = 0;
    virtual void sendPush(const User&, const std::string&) = 0;
};
```

If that was the case,

This fat interface defines methods for every channel. If we try to implement it in a class like **EmailSenderService**, which is only supposed to send emails, we'd end up with something like:

```
class EmailSenderService : public INotificationService {
public:
    void sendEmail(const User& u, const std::string& msg) override {
        // send email
    }
    void sendSMS(const User& u, const std::string& msg) override {
        // Email service doesn't send SMS - what now?
    }
    void sendPush(const User& u, const std::string& msg) override {
        // Email service doesn't send Push notifications either
    }
};
```

Another example …

```
1
2  // "Fat" interface forces all machines to implement unused methods
3  struct IMachine {
4      virtual void print(Document& doc) = 0;
5      virtual void scan(Document& doc) = 0;
6      virtual void fax(Document& doc) = 0;
7      virtual ~IMachine() = default;
8  };
9
10 struct SimplePrinter : IMachine {
11     void print(Document& doc) override {
12         // actual printing logic
13     }
14     void scan(Document& doc) override {
15         // dummy - not supported
16         throw std::logic_error("Scan not supported");
17     }
18     void fax(Document& doc) override {
19         // dummy - not supported
20         throw std::logic_error("Fax not supported");
21     }
22 };
23
24
```

```cpp
// Segregated interfaces
struct IPrinter {
    virtual void print(Document& doc) = 0;
    virtual ~IPrinter() = default;
};

struct IScanner {
    virtual void scan(Document& doc) = 0;
    virtual ~IScanner() = default;
};

struct IFax {
    virtual void fax(Document& doc) = 0;
    virtual ~IFax() = default;
};

// A simple printer now only depends on IPrinter
struct SimplePrinter : IPrinter {
    void print(Document& doc) override {
        // printing logic here
    }
};
```

```
23
24 // A fax-only device
25 struct FaxMachine : IFax {
26     void fax(Document& doc) override {
27         // faxing logic here
28     }
29 };
30
31 // A multifunction device can compose or inherit multiple interfaces
32 struct MultiFunctionPrinter : IPrinter, IScanner, IFax {
33     void print(Document& doc) override {
34         // print logic
35     }
36     void scan(Document& doc) override {
37         // scan logic
38     }
39     void fax(Document& doc) override {
40         // fax logic
41     }
42 };
43
```

Ok, thats all…

Following ISP keeps our code flexible and implementers happy – each class only signs up for what it can actually do.

In our notification system, because we anticipated the need to separate concerns, we avoided creating one giant interface.

Instead, we opted for one role per interface (INotifier for sending a notification via one channel).

This way, each notifier class has a minimal interface to implement, which it does fully. We don’t have unused methods hanging around in our classes.

SOLID Principles

Episode 6 - DIP

What is this new thing - DIP ???

The Dependency Inversion Principle states two key things:

1. High-level modules should not depend on low-level modules; both should depend on abstractions.
2. Abstractions should not depend on details; details should depend on abstractions.

In our notification system,

Originally NotificationService (high-level policy for notifying) was tightly coupled to concrete low-level implementations (std::cout output for email/SMS).

We improved that by introducing INotifier.

Now Notification Service can be seen as a high-level module, and EmailSender/SMSSender are low-level implementations.

Version Old

```cpp
class EmailSender {
public:
    void sendEmail(const User& user, const std::string& message) {
        // Only handles email-sending details
        std::cout << "[Email] To: " << user.email
                  << " | Message: " << message << "\n";
    }
};

class SMSSender {
public:
    void sendSMS(const User& user, const std::string& message) {
        // Only handles SMS-sending details
        std::cout << "[SMS] To: " << user.phone
                  << " | Message: " << message << "\n";
    }
};
```

Version Old

```
18
19 class NotificationService {
20 private:
21     EmailSender emailSender;
22     SMSSender smsSender;
23 public:
24     void sendNotification(const User& user, const std::string& message,
   const std::string& type) {
25         // Delegates the work to the appropriate sender
26         if (type == "email") {
27             emailSender.sendEmail(user, message);
28         } else if (type == "sms") {
29             smsSender.sendSMS(user, message);
30         }
31     }
32 };
```

Version Old

```
18
19 class NotificationService {
20 private:
21     EmailSender emailSender;
22     SMSSender smsSender;
23 public:
24     void sendNotification(const User& user, const std::string& message,
   const std::string& type) {
25         // Delegates the work to the appropriate sender
26         if (type == "email") {
27             emailSender.sendEmail(user, message);
28         } else if (type == "sms") {
29             smsSender.sendSMS(user, message);
30         }
31     }
32 };
```

```cpp
// Abstract base class (interface) for notification senders
struct INotifier {
    virtual ~INotifier() = default;
    virtual void send(const User& user, const std::string& message) = 0;
};

class EmailSender : public INotifier {
public:
    void send(const User& user, const std::string& message) override {
        std::cout << "[Email] To: " << user.email << " | Message: " << message << "\n";
    }
};

class SMSSender : public INotifier {
public:
    void send(const User& user, const std::string& message) override {
        std::cout << "[SMS] To: " << user.phone << " | Message: " << message << "\n";
    }
};
```

```cpp
class NotificationService {
public:
    // Send a notification using any notifier (polymorphism)
    void sendViaChannel(INotifier& notifier, const User& user, const
std::string& message) {
        notifier.send(user, message);
    }
};


User alice{"Alice", "alice@example.com", "1234567890"};
NotificationService service;
EmailSender emailSender;
SMSSender smsSender;

service.sendViaChannel(emailSender, alice, "Hello via Email!");
service.sendViaChannel(smsSender, alice, "Hello via SMS!");

```

We've inverted the dependency: instead of

NotificationService -> EmailSender (which was the initial design),

we have

NotificationService -> INotifier and

EmailSender -> INotifier.

The direction is inverted in terms of abstraction (high-level depends on interface, not concrete class).

With the SOLID principles applied, our design is significantly improved:

SRP: Each class has a single responsibility.

OCP: The system can be extended with new notifiers without modifying existing code.

LSP: Subclasses (notifiers) can substitute for the base (INotifier) reliably.

ISP: Interfaces are small and focused (just one method here).

DIP: High-level logic depends on an abstraction (INotifier), not concrete classes.

Another example??
ok…
let’s see what
we can do here

```cpp
class FileManager {
public:
    std::string fileName;
    std::vector<std::string> data;

    FileManager(const std::string& fileName) : fileName(fileName) {}

    void read() {
        std::cout << "Reading file " << fileName << std::endl;
        // File reading logic here
    }

    void write() {
        std::cout << "Writing to file " << fileName << std::endl;
        // File writing logic here
    }

    void processCSV() {
        std::cout << "Processing CSV file " << fileName << std::endl;
        // CSV processing logic
    }

    void processJSON() {
        std::cout << "Processing JSON file " << fileName <<
std::endl; // JSON processing logic
    }
};

void handleFiles(FileManager* fm) {
    fm->read();
    fm->processCSV();  // Violates Liskov Substitution Principle
    fm->write();
}

int main() {
    FileManager fm("example.csv");
    handleFiles(&fm);

    return 0;
}
```

Ok, now we understand… but…

SOLID Principles

Episode 7 - The Right Mental Model!

Framework for SOLID - the VG way.

Theory doesn’t result in Outcomes.

Actionable Points needed!

- What is wrong?
- How can we fix?

```java
class Invoice {
    double total;
    String customerEmail;

    void addItem(String name, double price) { total += price; }

    double applyDiscount(String coupon) {
        if ("NEW50".equals(coupon)) return total * 0.5;
        if ("FEST10".equals(coupon)) return total * 0.9;
        return total;
    }

    void saveToDb() {
        System.out.println("INSERT INTO invoices ...");
    }

    void sendEmail() {
        System.out.println("Sending email to " + customerEmail);
    }
}

```

Detection Framework : What do we look for Violation?

SRP: ??

OCP: ??

LSP: ??

ISP: ??

DIP: ??

SRP

```java
class Invoice {
    double total;
    String customerEmail;

    void addItem(String name, double price) { total += price; }

    double applyDiscount(String coupon) {
        if ("NEW50".equals(coupon)) return total * 0.5;
        if ("FEST10".equals(coupon)) return total * 0.9;
        return total;
    }

    void saveToDb() {
        System.out.println("INSERT INTO invoices ...");
    }

    void sendEmail() {
        System.out.println("Sending email to " + customerEmail);
    }
}

```

Detection Framework : What do we look for Violation?

SRP: Classes having different responsibilities. Too Long code in a Function.

OCP: ??

LSP: ??

ISP: ??

DIP: ??

OCP

```java
class Invoice {
    double total;
    String customerEmail;

    void addItem(String name, double price) { total += price; }

    double applyDiscount(String coupon) {
        if ("NEW50".equals(coupon)) return total * 0.5;
        if ("FEST10".equals(coupon)) return total * 0.9;
        return total;
    }

    void saveToDb() {
        System.out.println("INSERT INTO invoices ...");
    }

    void sendEmail() {
        System.out.println("Sending email to " + customerEmail);
    }
}
```

Detection Framework : What do we look for Violation?

SRP: Functions having different responsibilities. Too Long code in a Function.

OCP: “IF-ELSE” cannot be there (except if it explicitly does only IF-ELSE).

LSP: ??

ISP: ??

DIP: ??

What about this?

```cpp
#include <stdexcept>
#include <string>
struct Rectangle {
    virtual void setWidth(int w){ width = w; }
    virtual void setHeight(int h){ height = h; }
    int area() const { return width * height; }
protected:
    int width{0}, height{0};
};

struct Square : Rectangle {
    void setWidth(int w) override { width = height = w; }
    void setHeight(int h) override { width = height = h; }
};

int main() {
    Rectangle* r = new Square();
    r->setWidth(4);
    r->setHeight(5); // caller expects 4x5, but becomes 5x5
    return r->area(); // surprising!
}
```

Detection Framework : What do we look for Violation?

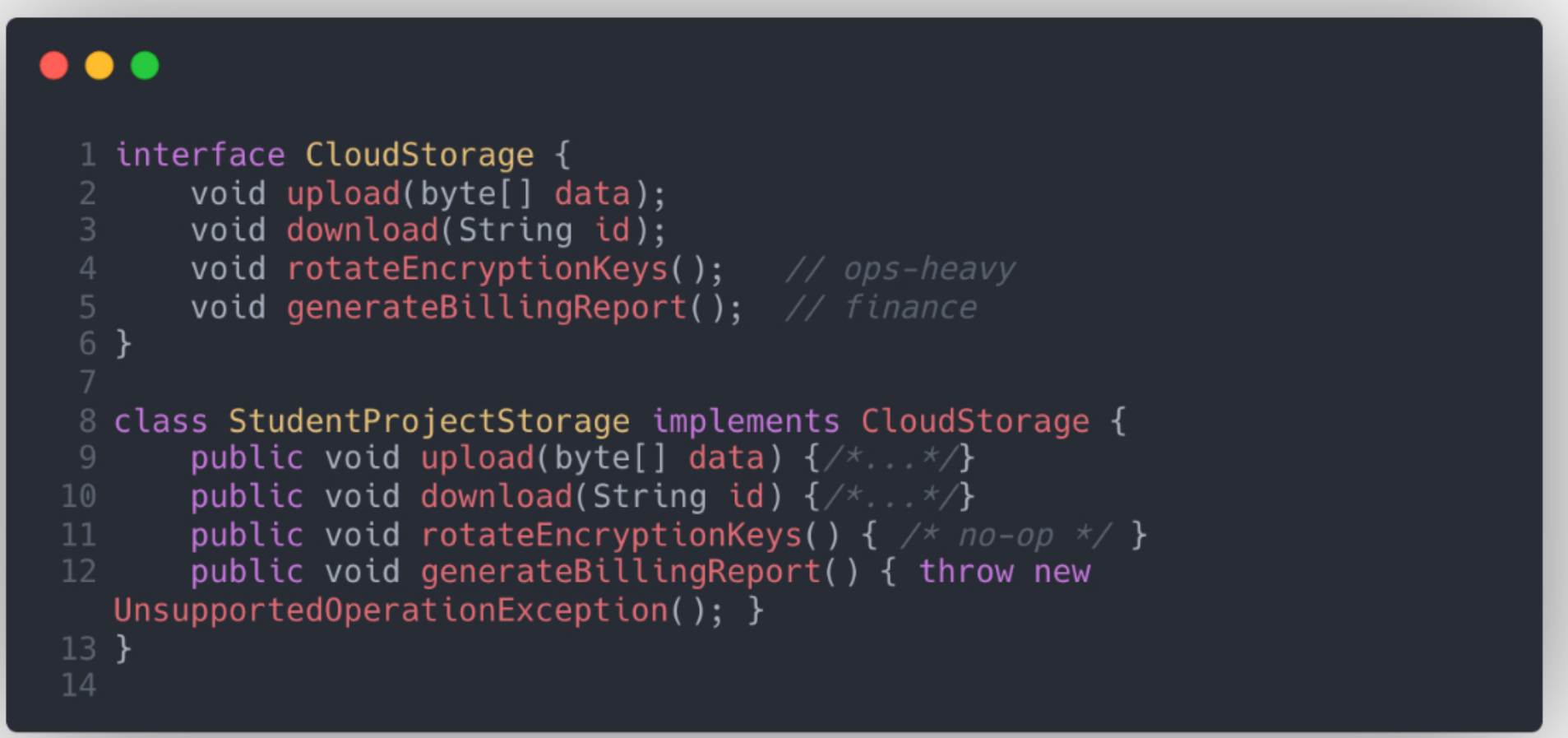
SRP: Functions having different responsibilities. Too Long code in a Function.

OCP: “IF-ELSE” cannot be there (except if it explicitly does only IF-ELSE).

LSP: Look for forced Inheritance and Overrides!

ISP: ??

DIP: ??



```java
interface CloudStorage {
    void upload(byte[] data);
    void download(String id);
    void rotateEncryptionKeys();   // ops-heavy
    void generateBillingReport();  // finance
}

class StudentProjectStorage implements CloudStorage {
    public void upload(byte[] data) {/*...*/}
    public void download(String id) {/*...*/}
    public void rotateEncryptionKeys() { /* no-op */ }
    public void generateBillingReport() { throw new UnsupportedOperationException(); }
}

```

Detection Framework : What do we look for Violation?

SRP: Functions having different responsibilities. Too Long code in a Function.

OCP: “IF-ELSE” cannot be there (except if it explicitly does only IF-ELSE).

LSP: Look for forced Inheritance and Overrides!

ISP: Bulky Interfaces… Too many functions to implement.

DIP: ??

```python
class MySqlDB:
    def save(self, record): print("Saving to MySQL", record)

class UserService:
    def __init__(self):
        self.db = MySqlDB()   # concrete dependency

    def register(self, user):
        # ... validations ...
        self.db.save(user)
```

Detection Framework : What do we look for Violation?

SRP: Functions having different responsibilities. Too Long code in a Function.

OCP: “IF-ELSE” cannot be there (except if it explicitly does only IF-ELSE).

LSP: Look for forced Inheritance and Overrides!

ISP: Bulky Interfaces… Too many functions to implement.

DIP: Object Static Creation -> Object Passed as Params -> Interface Type

```cpp
class FileManager {
public:
    std::string fileName;
    std::vector<std::string> data;

    FileManager(const std::string& fileName) : fileName(fileName) {}

    void read() {
        std::cout << "Reading file " << fileName << std::endl;
        // File reading logic here
    }

    void write() {
        std::cout << "Writing to file " << fileName << std::endl;
        // File writing logic here
    }

    void processCSV() {
        std::cout << "Processing CSV file " << fileName << std::endl;
        // CSV processing logic
    }

    void processJSON() {
        std::cout << "Processing JSON file " << fileName <<
std::endl; // JSON processing logic
    }
};

void handleFiles(FileManager* fm) {
    fm->read();
    fm->processCSV();  // Violates Liskov Substitution Principle
    fm->write();
}

int main() {
    FileManager fm("example.csv");
    handleFiles(&fm);

    return 0;
}
```

VG SOLID framework — Quick scan heuristics

SRP — Functions having different responsibilities. Too Long code in a Function.

OCP — "IF-ELSE" cannot be there (except if it explicitly does only IF-ELSE).

LSP — Look for forced Inheritance and Overrides!

ISP — Bulky Interfaces... Too many functions to implement

DIP — Object Static Creation → Object Passed as Params → Interface Type

```cpp
// Interface to follow Dependency Inversion Principle
class IFileProcessor {
public:
    virtual void process() = 0;
};

// Interface segregation for read and write operations
class IFileReader {
public:
    virtual void read() = 0;
};

class IFileWriter {
public:
    virtual void write() = 0;
};

// Separate class for reading
class FileReader : public IFileReader {
public:
    std::string fileName;

    FileReader(const std::string& fileName) : fileName(fileName) {}

    void read() override {
        std::cout << "Reading file " << fileName << std::endl;
        // File reading logic here
    }
};

// Separate class for writing
class FileWriter : public IFileWriter {
public:
    std::string fileName;

    FileWriter(const std::string& fileName) : fileName(fileName) {}

    void write() override {
        std::cout << "Writing to file " << fileName << std::endl;
        // File writing logic here
    }
};

```

```cpp
// Class for CSV processing, implements IFileProcessor
class CSVProcessor : public IFileProcessor {
public:
    void process() override {
        std::cout << "Processing CSV file" << std::endl;
        // CSV processing logic here
    }
};

// Class for JSON processing, implements IFileProcessor
class JSONProcessor : public IFileProcessor {
public:
    void process() override {
        std::cout << "Processing JSON file" << std::endl;
        // JSON processing logic here
    }
};

void handleFiles(IFileReader* reader, IFileProcessor* processor, IFileWriter* writer)
{   reader->read();
    processor->process();
    writer->write();
}

int main() {
    FileReader reader("example.csv");
    FileWriter writer("example.csv");
    CSVProcessor csvProcessor;

    handleFiles(&reader, &csvProcessor, &writer);

    return 0;
}

```

Weekly Assignments

Starter Code :

https://github.com/Vivekgupta98/lld-camp-week1-assignment

Checking Link:

https://shorturl.at/Pro0f